Mixed Precision Iterative Refinement and Approximate Hardware

Nico Teufel

Feb 20, 2026

Abstract

The following report summarizes in chronological order the activities and results of the beginner's practical course (IAP) at Heidelberg University. The report is structured in two main sections: the first focuses on the implementation and analysis of Mixed Precision Iterative Refinement (MPIR) for solving linear systems, while the second delves into the exploration of approximate hardware for multiplication.

1 Mixed Precision Iterative Refinement

In modern numerical computing, the primary bottleneck has shifted from raw floating-point performance to the overhead of data movement. Mixed Precision Iterative Refinement (MPIR) addresses this by pairing high- and low-precision operations to achieve solutions at full working accuracy. By exploiting the reduced storage and bandwidth requirements of lower-precision formats, MPIR facilitates faster data transfers and reduces overall execution time. The theoretical framework for these multi-precision approaches was notably established by Carson and Higham [1].

1.1 Mathematical Framework

1.1.1 Precision

We define the machine epsilon, μ, as the maximum relative distance between a real number and its nearest floating-point representation. An operation in a specific precision u is defined by the standard model $fl(x \text{ op } y) = (x \text{ op } y)(1 + \delta)$ with $|\delta| \leq u$, where u acts as the relative error bound for every intermediate calculation. In the context of MPIR, we utilize several levels of precision: μ_r (residual), μ_f (factorization), μ_s (solve), and μ (working).

1.1.2 Iterative Refinement

For a linear system $Ax = b$, the iterative refinement process starting from an initial solution x_0 consists of the following steps for $i = 0, 1, 2, \ldots$:

1. Compute the residual $r_i = b - Ax_i$ in precision u_r (residual precision).
2. Solve $Ad_i = r_i$ for the correction d_i using a pre-computed factorization performed in precision u_f (factorization precision), with the solve itself performed in precision u_s (solve precision).
3. Update the solution $x_{i+1} = x_i + d_i$ in working precision u.

1.1.3 Convergence & Stability

The convergence and stability of the iterative refinement process is described in detail in the work of Carson and Higham (2018) and can be looked up there.

1.2 Implementation for the 1D-Poisson Problem

The system was implemented for the 1D-Poisson Problem with a constant right-hand side (the Laplace equation). This discretization results in a SPD matrix, allowing for the use of a Cholesky factorization in the solver step. To evaluate the empirical accuracy of the iterative refinement against the theoretical bounds, a reference solution x_{ref} was calculated by solving the linear system directly in double precision. This allows for a direct comparison between the observed relative error and the theoretical forward error bound:

$$\frac{\|x_{ref} - \hat{x}\|_\infty}{\|x_{ref}\|_\infty}$$

1.3 Results

The iterative refinement algorithm was evaluated with several precision configurations to observe the impact on convergence and the limiting forward error. The configurations are denoted as (u_r, u, u_s, u_f), where the numbers represent the bit-width (64 for double, 32 for single, 16 for half precision).

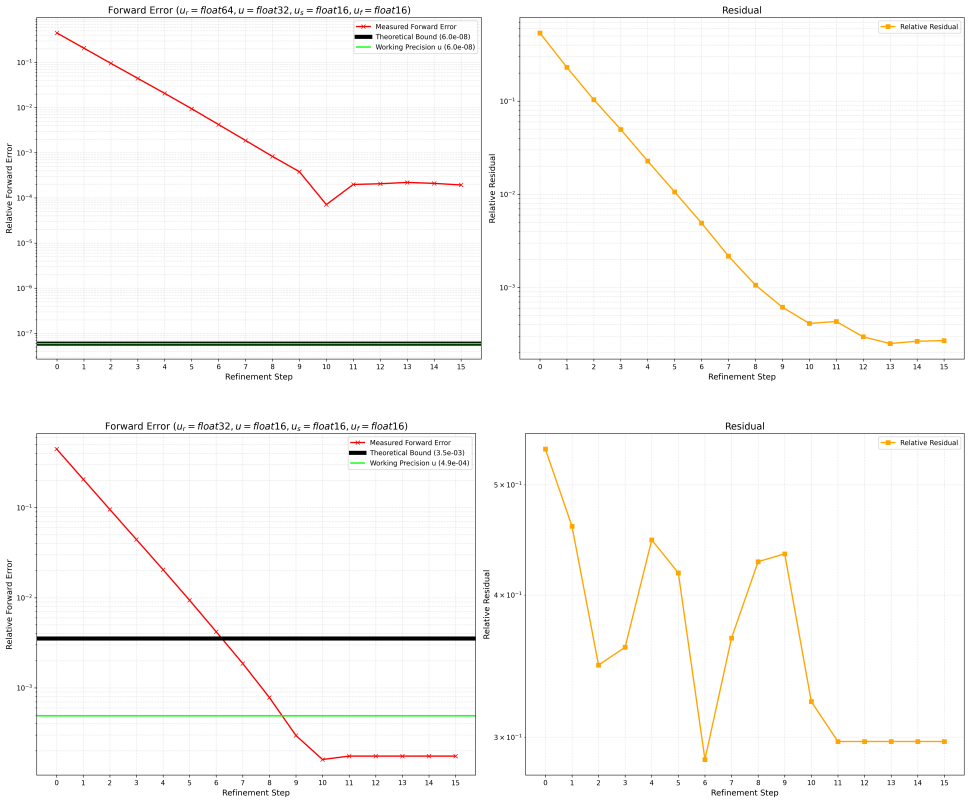

Figure 1: Comparison of mixed precision configurations. Top: $(64, 32, 16, 16)$ shows stagnation as u_s = float16 is insufficient. Bottom: $(32, 16, 16, 16)$ shows the refinement process when the working precision is already reduced to float16.

The results confirm that while the factorization can be performed in lower precision (u_f = float16), the solve precision u_s must be sufficient to guarantee convergence. For the 1D-Poisson problem with $N = 100$ ($\text{cond}(A) \approx 4 \times 10^3$), u_s = float32 is required.

2 Approximate Hardware for Multiplication

During a research meeting at ZITI (Institute of Computer Engineering) in Heidelberg, topics at the intersection of machine learning, numerical analysis, and computer engineering were discussed. This project investigates a hardware simulation provided by ZITI to explore approximate computing. The objective is to analyze the uncertainty introduced by approximate components. Empirical experiments were conducted and a formal mathematical description and a corresponding computational savings model were developed.

2.1 Statistical Experiments

To understand the behavior of the different approximation hardware better, the relative error distribution was investigated. This approach proved useful for real world scenarios, where the distinction between the machine accuracy and the approximation accuracy is not relevant.

The error characteristics were analyzed empirically by conducting a large-scale statistical simulation. For each hardware configuration (defined by its chromosome), 10.0M pairs of operands (a, b) were independently sampled from a uniform distribution $U(-1000, 1000)$. For each pair, the approximate product $\text{approx}(a, b)$ was computed using the hardware simulation and compared against a reference floating-point product $\text{ref}(a, b) = a \cdot b$ calculated in double precision. The relative error was then calculated as:

$$\epsilon_{rel} = \frac{\text{approx}(a, b) - \text{ref}(a, b)}{\text{ref}(a, b)} \tag{1}$$

By a large number of samples, the approximate multiplier reveals a systematic bias towards undershooting.

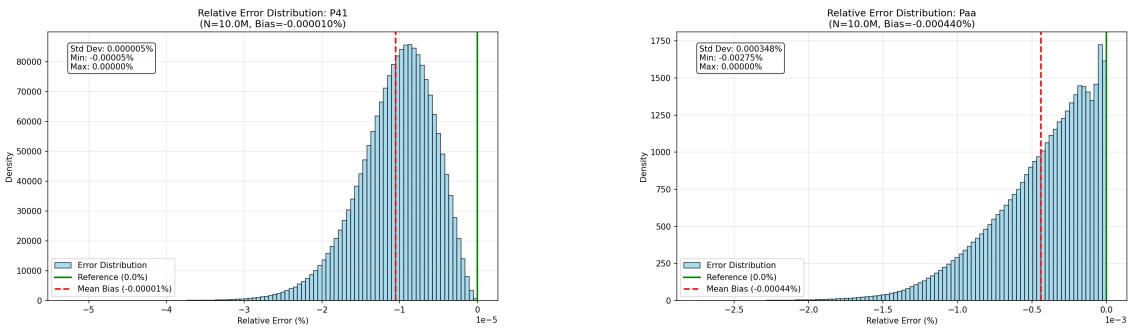

Figure 2: Relative error distribution for two approximate multiplier configurations ZITI provided. In particular, the left configuration was the weakest approximation, while the right configuration was the strongest approximation.

2.2 Mathematical Description of the Approximation Process

Understanding the code simulating the approximate hardware provided a challenge. The following section provides a mathematical description of the approximation process, abstracting away from implementation details. Note that the exponents are not approximated at all, they are calculated exactly. The approximation only happens in the mantissa and for 32-bit integers.

Let A and B be the mantissas of the 32-bit integer. That means they are 24-bit integers. Both are divided into three 8-bit segments via shifts, resulting in:

$$A = A_H \cdot 2^{16} + A_M \cdot 2^8 + A_L \cdot 2^0 \tag{2}$$

and equivalent for B.

By law of distribution, the multiplication AB results in the summation of nine 8×8 bit multiplications, so called partial products. In notation:

$$\begin{aligned} A \times B = A_H B_H \cdot 2^{32} &+ (A_H B_M + A_M B_H) \cdot 2^{24} \\ &+ (A_H B_L + A_M B_M + A_L B_H) \cdot 2^{16} \\ &+ (A_M B_L + A_L B_M) \cdot 2^{8} + A_L B_L \cdot 2^{0} \end{aligned} \tag{3}$$

We refer to $A_x B_y$ in a partial product as head of the partial product. Thus each pair of indices can be mapped to a power of 2^k. A partial product has therefore the form of $AB_{xy} := A_x B_y \cdot 2^{k(x,y)}$ for a $k(x, y) \in \mathbb{N}$ depending on the indices x and y. A total preorder for the partial products is established the following way.

$$AB_{xy} \geq AB_{wz} \iff k(x, y) \geq k(w, z) \tag{4}$$

A strict total order can be established by imposing a formal order on the elements that satisfy k(x,y) = k(w,z). This results in a strict total order which of course depends on the order imposed. The code utilizes the following order:

Table 1: Established order of partial products for chromosome mapping.

Index	**Element**	**Weight** (2^k)
0	$A_L B_L$	2^0
1	$A_M B_L$	2^8
2	$A_L B_M$	2^8
3	$A_H B_L$	2^{16}
4	$A_M B_M$	2^{16}
5	$A_L B_H$	2^{16}
6	$A_H B_M$	2^{24}
7	$A_M B_H$	2^{24}
8	$A_H B_H$	2^{32}

Each head of the partial product is further subdivided into two 8×4 bit products, referred to as **nibble products**. Again we say head of the nibble product. An order is again established based on powers of two.

A so-called **chromosome** defines the approximation intensity. It is structured as follows:

- It is a list of 9 natural numbers in the range $[0, 100]$.
- 100 represents the strongest approximation, 0 the lowest.
- The positions in the list have a natural one-to-one mapping to the partial products with the *importance order* established above, starting from the smallest element.

This means C_i in the chromosome is used as a configuration to approximate the i-th 8×8 product (bottom-up; least significant product first).

For $C_i < 100$, the configuration value has a representation $C = 10a + b$ with unique $a, b \in \{0, ..., 9\}$. We define the approximation factors X and Y as:

$$X := a + 1 \quad \text{and} \quad Y := b + 1$$

The **high-relevance nibble product** is approximated with intensity Y, while the **low-relevance nibble product** is approximated with intensity X.

For the special case $C_i = 100$, the entire 8×8 partial product is zeroed out ($AB_{xy} = 0$), representing the maximum possible approximation.

For an 8×4 nibble product using standard base-2 long multiplication, the process results in the summation of four rows. An approximation of intensity W for the nibble product means the following:

1. The least-significant W columns are approximated.
2. The addition operation in those columns is replaced by an **OR gate**.
3. Consequently, only a single entry needs to be 1 for the column sum to be 1.
4. This eliminates carry bits for the subsequent column, resulting exclusively in underestimation.

To obtain the final result of the 24×24 bit approximation, the nibble products are summed up, followed by the partial products.

Since additions occur at different layers, they can also be approximated. However, for simplicity, these additions are treated as exact here; approximating them would require extending the presented model.

Further, this description allows for finding bounds for the error of the approximate hardware.

2.3 Cost Savings Model

To evaluate the efficiency of the approximate hardware, a simple **cost-savings model** was developed. Instead of calculating the absolute gate count, the model quantifies the savings, defined as the number of addition operations replaced by simple OR-gates.

The savings are calculated at the lowest layer of the multiplication, specifically within the 8×4 **nibble products**. As established in the previous section, the approximation intensity W for a nibble product is determined by the factors X or Y.

A standard 8×4 nibble product is calculated by summing four rows, where each row is shifted by one bit relative to the previous one. This results in 11 columns with varying heights. The column height represents the number of bits to be summed in that specific position. The structure can be visualized as follows (where each • represents a bit):

	Columns (MSB ← LSB)										
	10	9	8	7	6	5	4	3	2	1	0
Row 0:				•	•	•	•	•	•	•	•
Row 1:			•	•	•	•	•	•	•	•	
Row 2:		•	•	•	•	•	•	•	•		
Row 3:	•	•	•	•	•	•	•	•			
Height:	1	2	3	4	4	4	4	4	3	2	1

The number of full-adders required in an exact implementation is proportional to the number of additions needed to resolve these column heights. By approximating a nibble product with intensity W, we replace the additions in the W least-significant columns with OR-gates. The total savings $S(W)$ are pre-calculated based on the cumulative reduction in required addition operations (see Table 2).

Table 2: Lookup table for savings $S(W)$ per intensity W (based on cumulative column height reduction).

Approximation Intensity W	**Approximated Columns**	**Savings**
1	0	0
2	1	1
3	2	3
4	3	6
5	4	9
6	5	12
7	6	15
8	7	18
9	8	20
10	9	21

The total savings for a specific chromosome entry C_i are calculated by utilizing the decoded intensities X and Y:

For $C_i < 100$:

$$\text{Savings}(C_i) = S(X) + S(Y) \tag{5}$$

For the special case $C_i = 100$:

$$\text{Savings}(100) = 2 \cdot S(10) = 42 \tag{6}$$

Finally, the total savings for a chromosome is the summation of the savings of all its entries.

Example: Uniform Chromosome

Consider the uniform chromosome $[41, 41, 41, 41, 41, 41, 41, 41, 41]$, where every entry equals $C_i = 41$.

For $C_i = 41 < 100$, write $C_i = 10 \cdot a + b$ with $a = 4$, $b = 1$, giving approximation intensities $X = a + 1 = 5$ and $Y = b + 1 = 2$. Looking up Table 2 yields $\text{Savings}(41) = S(5) + S(2) = 9 + 1 = 10$ per entry. Since the chromosome has 9 entries, all equal to 41, the total savings are:

$$\text{Total Savings} = \sum_{i=0}^{8} \text{Savings}(C_i) = 9 \times 10 = 90 \tag{7}$$

3 Further Research

These results make it possible to work out upper and lower bounds for different chromosomes. With the savings model, the path is paved to define a quotient of cost and approximation quality. This in turn would help with a fitness function for an evolutionary process to find the best chromosome based on the savings and approximation quality, now that a savings model is established.

References

[1] Carson, E., and Higham, N. J. (2018). Accelerating the solution of linear systems by iterative refinement in three precisions. *SIAM Journal on Scientific Computing*, 40(2), A817–A847.